

**UNIVERSITAS ESA UNGGUL**



**IMPLEMENTASI SISTEM BASIS DATA  
TERDISTRIBUSI DENGAN REPLIKASI MASTER-  
SLAVE DAN DASHBOARD MONITORING BERBASIS  
WEB UNTUK JURNAL TRADING**

**LAPORAN PROYEK AKHIR**

Diajukan sebagai salah satu syarat penilaian mata kuliah Sistem Basis Data  
Terdistribusi (SBDT)

**NAMA : DEBY HENDRI YARTO**

**NIM : 20220801294**

**PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS ILMU KOMPUTER  
UNIVERSITAS ESA UNGGUL  
TAHUN 2026**

## KATA PENGANTAR

Puji dan syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan proyek akhir yang berjudul “Implementasi Sistem Basis Data Terdistribusi dengan Replikasi Master-Slave dan Dashboard Monitoring Berbasis Web untuk Jurnal Trading” ini dapat diselesaikan dengan baik guna memenuhi salah satu syarat penilaian mata kuliah Sistem Basis Data Terdistribusi (SBDT) pada Program Studi Teknik Informatika, Fakultas Ilmu Komputer, Universitas Esa Unggul.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, dukungan, dan bantuan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada Dosen Pengampu mata kuliah Sistem Basis Data Terdistribusi atas arahan dan masukan yang sangat berharga selama proses penyusunan studi kasus dan implementasi sistem ini, serta kepada seluruh pihak yang telah membantu baik secara langsung maupun tidak langsung.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan dan jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami penerapan replikasi basis data terdistribusi pada studi kasus aplikasi jurnal trading.

Jakarta, Juli 2026

**Deby Hendri Yarto**

## ABSTRAK

Nama : Deby Hendri Yarto

NIM : 20220801294

Judul : Implementasi Sistem Basis Data Terdistribusi dengan Replikasi Master-Slave dan Dashboard Monitoring Berbasis Web untuk Jurnal Trading

Pencatatan transaksi jurnal trading secara harian menghasilkan volume data yang terus bertambah, sementara kebutuhan untuk membaca data tersebut melalui dashboard monitoring dapat membebani server basis data yang sama apabila tidak dipisahkan dari proses transaksi. Penelitian ini bertujuan membangun sistem basis data terdistribusi dengan skema replikasi Master-Slave secara asynchronous menggunakan MySQL yang di-deploy pada container Docker, guna memisahkan beban tulis (write) pada server Master dan beban baca (read) pada server Slave. Aplikasi pencatatan jurnal trading dibangun menggunakan framework Laravel dengan panel admin Filament, sedangkan dashboard monitoring performa trading membaca data secara khusus dari server Slave. Metode pengembangan sistem yang digunakan adalah Prototyping, dengan tahapan analisis kebutuhan, perancangan arsitektur dan basis data, implementasi, serta pengujian. Pengujian dilakukan melalui verifikasi status replikasi (SHOW SLAVE STATUS), pengujian konsistensi data antar server, dan pengujian fungsional black-box terhadap aplikasi. Hasil pengujian menunjukkan bahwa replikasi Master-Slave berhasil menjaga konsistensi data secara otomatis antara Master dan Slave, serta pemisahan beban baca ke Slave terbukti mengurangi beban komputasi pada server Master. Sistem yang dibangun membuktikan bahwa arsitektur basis data terdistribusi dapat diterapkan pada skala aplikasi web sederhana untuk meningkatkan ketersediaan (availability) dan keandalan data.

*Kata Kunci: basis data terdistribusi, replikasi master-slave, docker, laravel filament, jurnal trading*

## ABSTRACT

Name : Deby Hendri Yarto

NIM : 20220801294

Title : Implementation of a Distributed Database System with Master-Slave Replication and a Web-Based Monitoring Dashboard for a Trading Journal

Daily recording of trading journal transactions produces a continuously growing volume of data, while the need to read that data through a monitoring dashboard can burden the same database server if it is not separated from the transactional workload. This study aims to build a distributed database system with an asynchronous Master-Slave replication scheme using MySQL deployed on Docker containers, in order to separate the write workload on the Master server from the read workload on the Slave server. The trading journal recording application was built using the Laravel framework with the Filament admin panel, while the trading performance monitoring dashboard reads data exclusively from the Slave server. The system development method used is Prototyping, consisting of requirements analysis, architecture and database design, implementation, and testing. Testing was carried out through replication status verification (SHOW SLAVE STATUS), data consistency testing between servers, and functional black-box testing of the application. The test results show that Master-Slave replication successfully maintains data consistency automatically between the Master and Slave, and that separating the read workload to the Slave proves effective in reducing computational load on the Master server. The system built demonstrates that a distributed database architecture can be applied at the scale of a simple web application to improve data availability and reliability.

*Keywords: distributed database, master-slave replication, docker, laravel filament, trading journal*

## DAFTAR ISI

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>KATA PENGANTAR.....</b>                                 | <b>i</b>   |
| <b>ABSTRAK .....</b>                                       | <b>ii</b>  |
| <b>ABSTRACT .....</b>                                      | <b>iii</b> |
| <b>DAFTAR ISI.....</b>                                     | <b>iv</b>  |
| <b>BAB 1 PENDAHULUAN .....</b>                             | <b>1</b>   |
| <b>1.1 Latar Belakang.....</b>                             | <b>1</b>   |
| <b>1.2 Rumusan Masalah.....</b>                            | <b>2</b>   |
| <b>1.3 Tujuan Penelitian.....</b>                          | <b>2</b>   |
| <b>1.4 Manfaat Penelitian .....</b>                        | <b>2</b>   |
| <b>1.4.1 Manfaat Teoritis.....</b>                         | <b>2</b>   |
| <b>1.4.2 Manfaat Praktis.....</b>                          | <b>3</b>   |
| <b>1.5 Batasan Masalah .....</b>                           | <b>3</b>   |
| <b>1.6 Sistematika Penulisan .....</b>                     | <b>3</b>   |
| <b>BAB 2 TINJAUAN PUSTAKA.....</b>                         | <b>5</b>   |
| <b>2.1 Studi Literatur.....</b>                            | <b>5</b>   |
| <b>2.2 Basis Data Terdistribusi .....</b>                  | <b>6</b>   |
| <b>2.3 Replikasi Basis Data.....</b>                       | <b>6</b>   |
| <b>2.3.1 Replikasi Asynchronous dan Synchronous.....</b>   | <b>6</b>   |
| <b>2.3.2 Skema Master-Slave .....</b>                      | <b>6</b>   |
| <b>2.4 Docker dan Containerization .....</b>               | <b>7</b>   |
| <b>2.5 Laravel dan Filament.....</b>                       | <b>7</b>   |
| <b>2.6 High Availability .....</b>                         | <b>7</b>   |
| <b>BAB 3 METODE DAN PERANCANGAN SISTEM.....</b>            | <b>8</b>   |
| <b>3.1 Metode Penelitian.....</b>                          | <b>8</b>   |
| <b>3.2 Analisis Kebutuhan.....</b>                         | <b>8</b>   |
| <b>3.2.1 Kebutuhan Fungsional .....</b>                    | <b>8</b>   |
| <b>3.2.2 Kebutuhan Non-Fungsional.....</b>                 | <b>9</b>   |
| <b>3.3 Perancangan Arsitektur Sistem .....</b>             | <b>9</b>   |
| <b>3.4 Perancangan Basis Data .....</b>                    | <b>10</b>  |
| <b>3.5 Perancangan Docker Compose .....</b>                | <b>10</b>  |
| <b>BAB 4 IMPLEMENTASI DAN HASIL.....</b>                   | <b>12</b>  |
| <b>4.1 Implementasi Konfigurasi Master dan Slave .....</b> | <b>12</b>  |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>4.2 Verifikasi Status Replikasi .....</b>              | <b>12</b> |
| <b>4.3 Implementasi Aplikasi dan Dashboard.....</b>       | <b>13</b> |
| <b>4.4 Pengujian Fungsional (Black-Box Testing) .....</b> | <b>14</b> |
| <b>4.5 Analisis Hasil Pengujian .....</b>                 | <b>15</b> |
| <b>BAB 5 KESIMPULAN DAN SARAN .....</b>                   | <b>17</b> |
| <b>5.1 Kesimpulan .....</b>                               | <b>17</b> |
| <b>5.2 Saran .....</b>                                    | <b>17</b> |
| <b>DAFTAR PUSTAKA .....</b>                               | <b>18</b> |

## BAB 1 PENDAHULUAN

### 1.1 Latar Belakang

Trading merupakan aktivitas jual beli instrumen keuangan yang membutuhkan pencatatan riwayat transaksi secara disiplin agar trader dapat mengevaluasi performa dan konsistensi strategi yang dijalankan. Pencatatan tersebut umumnya dituangkan dalam bentuk jurnal trading, yang berisi tanggal transaksi, pasangan instrumen (pair), hasil transaksi (win/loss), serta nominal profit atau loss yang diperoleh. Aktivitas pencatatan ini dilakukan secara rutin dan menghasilkan volume data yang terus bertambah seiring berjalannya waktu.

Permasalahan muncul ketika seluruh proses input data transaksi dan pembacaan data untuk keperluan analitik (dashboard monitoring) dibebankan pada satu server basis data yang sama. Ketika volume data dan frekuensi akses meningkat, query analitik yang kompleks dapat memperlambat proses input transaksi, bahkan berpotensi menyebabkan gangguan (downtime) pada saat server sedang sibuk. Selain itu, apabila server basis data mengalami kegagalan, seluruh proses pencatatan maupun pembacaan data akan terhenti secara bersamaan, sehingga data historis berisiko tidak dapat diakses.

Untuk mengatasi permasalahan tersebut, diperlukan arsitektur basis data terdistribusi yang mampu memisahkan beban kerja penulisan data (write) dan pembacaan data (read) pada server yang berbeda, namun tetap menjaga konsistensi data di antara keduanya. Salah satu pendekatan yang umum digunakan adalah replikasi Master-Slave, di mana server Master menangani seluruh operasi tulis, sedangkan server Slave menerima salinan data secara otomatis melalui mekanisme replikasi dan digunakan khusus untuk operasi baca. Pendekatan ini telah terbukti efektif pada berbagai studi kasus untuk meningkatkan ketersediaan (high availability) dan performa sistem berbasis MySQL yang dijalankan pada lingkungan container Docker.

Berdasarkan uraian tersebut, penelitian ini bermaksud mengimplementasikan sistem basis data terdistribusi dengan skema replikasi Master-Slave menggunakan

Docker, disertai dengan aplikasi jurnal trading berbasis web menggunakan Laravel dan Filament, serta dashboard monitoring yang membaca data secara khusus dari server Slave, sebagai studi kasus mata kuliah Sistem Basis Data Terdistribusi.

## 1.2 Rumusan Masalah

- Bagaimana mekanisme replikasi Master-Slave dapat menjamin konsistensi data jurnal trading antara server Master dan server Slave?
- Bagaimana memisahkan beban kerja input transaksi (write) pada Master dan beban kerja dashboard monitoring (read) pada Slave agar tidak saling mengganggu?
- Bagaimana Docker dapat digunakan untuk menyediakan lingkungan basis data terdistribusi yang mudah dikonfigurasi dan direplikasi pada server berbeda?

## 1.3 Tujuan Penelitian

- Membangun sistem basis data terdistribusi dengan skema replikasi Master-Slave secara asynchronous untuk aplikasi jurnal trading.
- Mengimplementasikan seluruh infrastruktur basis data dan aplikasi web dalam container Docker agar konfigurasi mudah direproduksi.
- Menyediakan dashboard monitoring performa trading berbasis web (Laravel Filament) yang membaca data secara khusus dari server Slave.
- Menguji konsistensi data serta performa sistem setelah skema replikasi diterapkan.

## 1.4 Manfaat Penelitian

### 1.4.1 Manfaat Teoritis

Penelitian ini diharapkan dapat memperkaya referensi penerapan konsep basis data terdistribusi, khususnya replikasi Master-Slave berbasis Docker, pada studi kasus aplikasi web skala kecil hingga menengah.



### 1.4.2 Manfaat Praktis

- Bagi pengguna aplikasi (trader), sistem ini menyediakan pencatatan jurnal trading yang tersimpan aman dengan ketersediaan data yang lebih tinggi.
- Bagi pengembang, arsitektur yang dibangun dapat dijadikan referensi implementasi replikasi basis data pada proyek serupa.
- Bagi institusi pendidikan, penelitian ini menjadi salah satu bentuk penerapan mata kuliah Sistem Basis Data Terdistribusi pada studi kasus nyata.

### 1.5 Batasan Masalah

Agar pembahasan lebih terarah, penelitian ini dibatasi pada ruang lingkup berikut:

- Replikasi basis data menggunakan MySQL dengan skema Master-Slave (satu Master dan satu Slave) secara asynchronous.
- Seluruh service (Master, Slave, dan aplikasi web) dijalankan menggunakan Docker dan Docker Compose.
- Aplikasi web dibangun menggunakan Laravel dengan panel admin Filament untuk pencatatan data (CRUD) jurnal trading.
- Dashboard monitoring menampilkan metrik performa trading (total trades, win rate, profit factor, net P/L) yang dibaca dari server Slave.
- Penelitian tidak membahas skema multi-master, cluster replikasi (Galera/NDB Cluster), load balancer tingkat lanjut, maupun strategi atau manajemen risiko trading itu sendiri.
- Pengujian dilakukan pada lingkungan pengembangan (local/Docker), bukan pada skala production dengan jumlah pengguna besar.

### 1.6 Sistematika Penulisan

Sistematika penulisan laporan ini disusun sebagai berikut. BAB 1 Pendahuluan menjabarkan latar belakang, rumusan masalah, tujuan, manfaat, dan batasan penelitian. BAB 2 Tinjauan Pustaka membahas studi literatur terdahulu dan

landasan teori yang mendasari penelitian. BAB 3 Metode dan Perancangan Sistem menjelaskan metodologi penelitian, analisis kebutuhan, serta perancangan arsitektur dan basis data. BAB 4 Implementasi dan Hasil menyajikan implementasi konfigurasi replikasi, aplikasi web, hasil pengujian, dan analisis hasil. BAB 5 Kesimpulan dan Saran memuat kesimpulan penelitian serta saran pengembangan lebih lanjut.

## BAB 2 TINJAUAN PUSTAKA

### 2.1 Studi Literatur

Beberapa penelitian terdahulu yang relevan dengan topik replikasi basis data dan penggunaan Docker dirangkum pada Tabel 2.1 berikut.

| Judul dan Sumber                                                                                                                                         | Tahun | Hasil Utama                                                                                                                                   | Relevansi                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Sholihah, I. — Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker. Majalah Ilmiah Teknologi Elektro.                             | 2022  | Replikasi Master-Slave pada MySQL berbasis Docker terbukti menyalin data secara konsisten antara server Master dan Slave.                     | Justifikasi teknik replikasi & Docker         |
| Penerapan Disaster Recovery Plan Pada Database MySQL Menggunakan Metode Master to Master Replication Berbasis Docker. Jurnal Komputer Terapan.           | 2024  | Replikasi database berbasis Docker efektif sebagai strategi pemulihan bencana (disaster recovery) data.                                       | Justifikasi ketersediaan data                 |
| Pamungkas, dkk. — Performance Evaluation of a Laravel-based Internship Website using MySQL Master-Slave Replication. Sistemasi: Jurnal Sistem Informasi. | 2026  | Konfigurasi Master-Slave pada aplikasi Laravel ber-container memberikan performa lebih stabil dibanding single database saat beban meningkat. | Justifikasi arsitektur Laravel + Master-Slave |
| High Availability and Performance of Database in the Cloud — Traditional Master-Slave Replication versus Modern Cluster-based Solutions.                 | 2017  | Replikasi Master-Slave tetap relevan untuk high availability dibandingkan solusi cluster modern seperti Galera.                               | Perbandingan pendekatan replikasi             |

Berdasarkan keempat studi tersebut, dapat disimpulkan bahwa replikasi Master-Slave berbasis Docker merupakan pendekatan yang telah banyak diterapkan dan terbukti mampu menjaga konsistensi data serta meningkatkan ketersediaan sistem. Namun demikian, sebagian besar penelitian terdahulu berfokus pada pengujian replikasi secara umum dan belum banyak yang mengintegrasikannya secara khusus dengan dashboard monitoring berbasis web pada studi kasus jurnal trading. Penelitian ini melengkapi kesenjangan tersebut dengan menerapkan replikasi

Master-Slave yang terintegrasi langsung dengan aplikasi Laravel Filament dan dashboard monitoring yang membaca data secara eksklusif dari server Slave.

## 2.2 Basis Data Terdistribusi

Basis data terdistribusi adalah kumpulan data yang secara logis saling terkait namun secara fisik tersebar pada beberapa lokasi atau server yang terhubung melalui jaringan komputer. Tujuan utama basis data terdistribusi adalah meningkatkan ketersediaan data, mempercepat akses data bagi pengguna pada lokasi berbeda, serta menyediakan mekanisme redundansi apabila salah satu server mengalami gangguan.

## 2.3 Replikasi Basis Data

Replikasi basis data adalah proses menyalin dan mensinkronkan data dari satu server basis data (Master) ke satu atau lebih server lain (Slave) secara otomatis. Replikasi bertujuan menjaga ketersediaan dan redundansi data, sehingga apabila server utama mengalami kegagalan, data tetap dapat diakses melalui server salinan.

### 2.3.1 Replikasi *Asynchronous* dan *Synchronous*

Replikasi asynchronous mengirimkan perubahan data dari Master ke Slave tanpa menunggu konfirmasi bahwa Slave telah selesai menerapkan perubahan tersebut, sehingga menghasilkan performa yang lebih baik untuk aplikasi web dengan konsekuensi kemungkinan adanya sedikit selisih waktu (replication lag) antara data di Master dan Slave. Sebaliknya, replikasi synchronous menunggu Slave selesai menulis data sebelum transaksi di Master dianggap selesai, sehingga menjamin konsistensi penuh namun dengan latensi lebih tinggi. Penelitian ini menggunakan skema asynchronous karena lebih sesuai dengan karakteristik aplikasi web yang mengutamakan responsivitas.

### 2.3.2 Skema *Master-Slave*

Pada skema Master-Slave, server Master bertindak sebagai satu-satunya server yang menerima operasi tulis (INSERT, UPDATE, DELETE), sedangkan server Slave bersifat read-only dan menerima perubahan data melalui binary log yang

dikirimkan oleh Master. Skema ini memungkinkan pemisahan beban kerja (read-write splitting), di mana operasi baca yang berat, seperti query dashboard analitik, dapat diarahkan ke Slave tanpa mengganggu performa Master.

## 2.4 Docker dan Containerization

Docker adalah platform containerization yang memungkinkan aplikasi beserta seluruh dependensinya dikemas dalam satu unit terisolasi yang disebut container. Docker Compose memungkinkan beberapa container (misalnya server basis data Master, Slave, dan aplikasi web) didefinisikan dan dijalankan bersama melalui satu berkas konfigurasi (docker-compose.yml), sehingga mencegah konflik dependensi (dependency hell) ketika aplikasi dijalankan pada lingkungan atau server yang berbeda.

## 2.5 Laravel dan Filament

Laravel adalah framework PHP berbasis arsitektur Model-View-Controller (MVC) yang menyediakan berbagai fitur bawaan seperti Eloquent ORM, routing, dan migration untuk mempercepat pengembangan aplikasi web. Filament adalah paket admin panel berbasis Laravel yang memungkinkan pembuatan antarmuka CRUD (Create, Read, Update, Delete) serta dashboard secara cepat dan reaktif, sehingga cocok digunakan untuk membangun panel input data jurnal trading maupun dashboard monitoring pada penelitian ini.

## 2.6 High Availability

High availability (ketersediaan tinggi) adalah karakteristik sistem yang dirancang agar tetap dapat diakses dan beroperasi meskipun terjadi gangguan pada salah satu komponennya. Dalam konteks basis data, high availability umumnya dicapai melalui replikasi data ke lebih dari satu server, sehingga apabila server utama mengalami kegagalan, server cadangan dapat melanjutkan layanan tanpa kehilangan data yang telah tersimpan.

## BAB 3 METODE DAN PERANCANGAN SISTEM

### 3.1 Metode Penelitian

Metode pengembangan sistem yang digunakan pada penelitian ini adalah Prototyping, yaitu sistem dibangun secara bertahap (Master, kemudian Slave, kemudian aplikasi dan dashboard) dengan pengujian pada setiap tahap sebelum dilanjutkan ke tahap berikutnya. Pendekatan ini dipilih karena memungkinkan penyesuaian konfigurasi replikasi secara cepat apabila ditemukan kendala pada tahap awal implementasi.

Tahapan penelitian yang dilakukan adalah sebagai berikut:

- Analisis Kebutuhan: menentukan kebutuhan fungsional (input transaksi, dashboard monitoring) dan non-fungsional (ketersediaan data, performa) dari sistem.
- Perancangan Sistem: merancang arsitektur replikasi, skema basis data, dan struktur berkas docker-compose.yml untuk setiap service.
- Implementasi: melakukan konfigurasi Master dan Slave, menjalankan container, serta membangun aplikasi Filament di atas Laravel.
- Pengujian dan Analisis: menguji konsistensi data antar server serta menganalisis performa dashboard yang mengakses Slave.

### 3.2 Analisis Kebutuhan

#### 3.2.1 Kebutuhan Fungsional

- Sistem dapat mencatat transaksi jurnal trading baru (tanggal, pair, hasil win/loss, dan nominal profit/loss) melalui panel admin Filament.
- Sistem dapat menampilkan dashboard monitoring yang berisi metrik total trades, win rate, profit factor, dan net P/L.
- Sistem dapat memfilter data dashboard berdasarkan rentang tanggal transaksi.

- Data yang dicatat pada server Master harus direplikasi secara otomatis ke server Slave.

### 3.2.2 Kebutuhan Non-Fungsional

- Ketersediaan (availability): data tetap dapat dibaca melalui dashboard meskipun server Master mengalami gangguan sementara.
- Performa: proses input transaksi pada Master tidak boleh terganggu oleh beban query dashboard pada Slave.
- Portabilitas: seluruh service dapat dijalankan pada server atau komputer berbeda tanpa konfigurasi manual yang rumit, menggunakan Docker Compose.

## 3.3 Perancangan Arsitektur Sistem

Arsitektur sistem yang dirancang terdiri dari lima komponen utama, yaitu pengguna (trader), aplikasi web Laravel Filament, server basis data Master, server basis data Slave, dan dashboard monitoring. Alur kerja sistem digambarkan sebagai berikut: pengguna berinteraksi dengan aplikasi Laravel Filament untuk mencatat transaksi baru. Seluruh operasi tulis diarahkan ke DB Master. Perubahan data pada Master direplikasi secara asynchronous ke DB Slave melalui binary log. Dashboard monitoring kemudian membaca data secara khusus dari DB Slave, sehingga beban query analitik tidak membebani proses transaksi pada Master.

| Komponen                   | Peran                                                                           |
|----------------------------|---------------------------------------------------------------------------------|
| User (Trader)              | Mengakses aplikasi web untuk mencatat dan memantau transaksi trading.           |
| Web App (Laravel Filament) | Menyediakan panel input data (CRUD) dan antarmuka dashboard.                    |
| DB Master (Write)          | Menerima seluruh operasi tulis (INSERT/UPDATE/DELETE) dari aplikasi.            |
| DB Slave (Read)            | Menerima replikasi data dari Master secara asynchronous; melayani operasi baca. |
| Dashboard Monitoring       | Menampilkan metrik performa trading berdasarkan data dari DB Slave.             |

Tabel 3.1 Komponen Arsitektur Sistem

Dengan alur komunikasi satu arah dari Master menuju Slave, sistem menjamin bahwa proses pencatatan transaksi (write) tetap responsif, sementara proses analitik (read) yang cenderung lebih berat dapat dilayani secara terpisah tanpa saling mengganggu.

### 3.4 Perancangan Basis Data

Tabel utama yang digunakan pada penelitian ini adalah tabel trades, yang menyimpan seluruh catatan transaksi jurnal trading dan direplikasi dari Master ke Slave. Struktur tabel trades dijabarkan pada Tabel 3.2.

| Kolom         | Tipe Data                   | Keterangan                                        |
|---------------|-----------------------------|---------------------------------------------------|
| id            | BIGINT (PK, auto increment) | Identitas unik tiap catatan transaksi trading     |
| tanggal       | DATE                        | Tanggal transaksi dilakukan                       |
| pair          | VARCHAR(20)                 | Pasangan instrumen yang ditradingkan              |
| hasil_wl      | ENUM('win','loss')          | Hasil transaksi: Win atau Loss                    |
| jumlah_rupiah | DECIMAL(15,2)               | Nominal profit/loss dalam Rupiah                  |
| created_at    | TIMESTAMP                   | Waktu data dicatat, dipakai untuk audit replikasi |

*Tabel 3.2 Struktur Tabel trades*

Prinsip perancangan skema basis data yang diterapkan adalah sebagai berikut: struktur tabel dibuat identik di Master dan Slave sebelum replikasi diaktifkan; hanya Master yang menerima perintah INSERT/UPDATE/DELETE; Slave bersifat read-only dan menerima perubahan melalui binary log Master; serta konsistensi data diverifikasi dengan membandingkan jumlah baris pada kedua server.

### 3.5 Perancangan Docker Compose

Seluruh service pada sistem, yaitu container db\_master, db\_slave, dan aplikasi web (app), didefinisikan dalam satu berkas docker-compose.yml. Setiap container database diberikan konfigurasi server-id yang berbeda (server-id: 1 untuk Master dan server-id: 2 untuk Slave) sebagai identitas unik dalam skema replikasi MySQL. Volume Docker digunakan untuk menyimpan data secara persisten, sedangkan



Docker network internal digunakan agar container Master, Slave, dan aplikasi dapat saling berkomunikasi tanpa perlu diekspos ke jaringan publik.

## BAB 4 IMPLEMENTASI DAN HASIL

### 4.1 Implementasi Konfigurasi Master dan Slave

Konfigurasi replikasi diterapkan melalui berkas `my.cnf` pada masing-masing container MySQL. Pada server Master, konfigurasi diarahkan untuk mengaktifkan binary logging sebagai berikut:

```
[mysqld] server-id=1 log_bin=/var/log/mysql/mysql-bin.log
binlog_do_db=jurnal_trading
```

Selanjutnya, pada server Master dibuat pengguna khusus replikasi beserta hak aksesnya, dan status binary log dicatat melalui perintah berikut:

```
CREATE USER 'replica'@'%' IDENTIFIED BY 'password'; GRANT
REPLICATION SLAVE ON *.* TO 'replica'@'%'; FLUSH PRIVILEGES;
SHOW MASTER STATUS;
```

Pada server Slave, konfigurasi `my.cnf` diarahkan untuk mengaktifkan relay log sebagai berikut:

```
[mysqld] server-id=2 relay-log=/var/log/mysql/mysql-relay-bin
```

Slave kemudian diarahkan untuk mengambil data dari Master menggunakan informasi log file dan posisi yang diperoleh dari perintah `SHOW MASTER STATUS` sebelumnya:

```
CHANGE MASTER TO MASTER_HOST='master_ip',
MASTER_USER='replica', MASTER_PASSWORD='password',
MASTER_LOG_FILE='...', MASTER_LOG_POS=...; START SLAVE;
```

### 4.2 Verifikasi Status Replikasi

Setelah konfigurasi diterapkan, status replikasi diverifikasi melalui perintah berikut yang dijalankan pada masing-masing container:

```
docker exec -it db_master mysql -u root -p -e "SHOW MASTER STATUS\G"
```

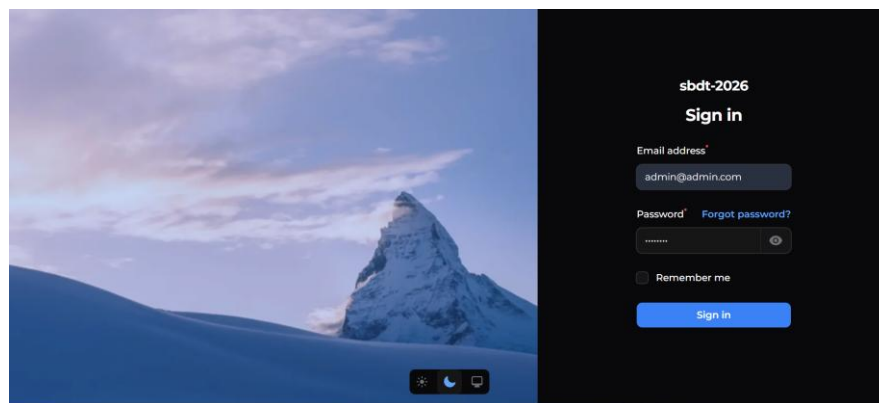
```
docker exec -it db_slave mysql -u root -p -e "SHOW SLAVE STATUS\G"
```

Indikator keberhasilan replikasi ditandai dengan nilai Slave\_IO\_Running: Yes dan Slave\_SQL\_Running: Yes pada output SHOW SLAVE STATUS, yang menandakan Slave berhasil menarik dan menerapkan data dari binary log Master secara otomatis.

### 4.3 Implementasi Aplikasi dan Dashboard

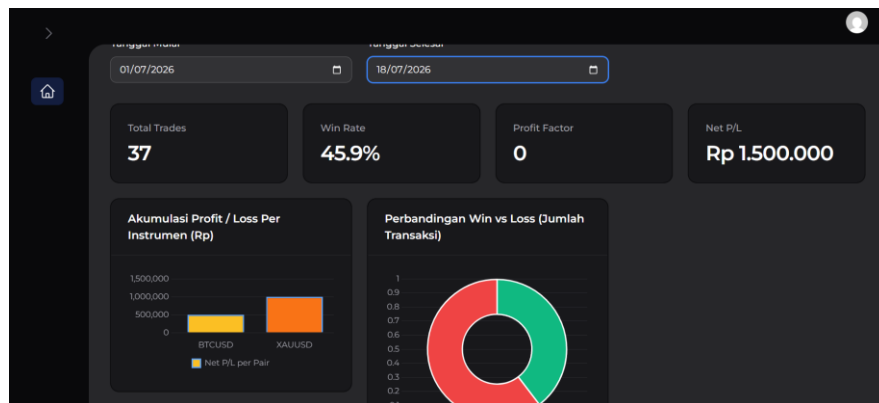
Aplikasi jurnal trading dibangun menggunakan Laravel dengan panel admin Filament. Panel admin (terhubung ke DB Master) digunakan untuk melakukan proses input dan pengelolaan data jurnal trading secara langsung melalui form “Catat Trade Cepat”, sedangkan panel dashboard (terhubung ke DB Slave) menampilkan rekap dan filter data historis tanpa membebani server Master. Perbedaan antara Master dan Slave pada konfigurasi Docker Compose dapat dilihat dari nilai server-id, yaitu server-id: 1 untuk Master dan server-id: 2 untuk Slave.

Gambar 4.1 hingga Gambar 4.3 berikut menunjukkan tampilan aplikasi yang telah berjalan.



Gambar 4.1 Halaman Login Panel Filament

Gambar 4.2 Form Input Jurnal Trading (“Catat Trade Cepat”)



Gambar 4.3 Dashboard Monitoring yang Membaca Data dari Slave

Berdasarkan Gambar 4.3, setelah data transaksi baru dicatat melalui form input, metrik pada dashboard — seperti Total Trades, Win Rate, dan Net P/L — diperbarui secara otomatis. Hal ini membuktikan bahwa data yang dicatat pada Master berhasil direplikasi ke Slave dan ditampilkan melalui dashboard tanpa memerlukan input manual tambahan pada sisi Slave.

#### 4.4 Pengujian Fungsional (Black-Box Testing)

Pengujian fungsional dilakukan untuk memastikan setiap fitur utama sistem berjalan sesuai dengan hasil yang diharapkan. Hasil pengujian black-box disajikan pada Tabel 4.1.

| Skenario Pengujian | Langkah                                | Hasil yang Diharapkan                | Status |
|--------------------|----------------------------------------|--------------------------------------|--------|
| Login admin        | Masukkan email dan password yang benar | Sistem menampilkan halaman Dashboard | Valid  |

| Skenario Pengujian                   | Langkah                                                    | Hasil yang Diharapkan                                         | Status |
|--------------------------------------|------------------------------------------------------------|---------------------------------------------------------------|--------|
|                                      | pada halaman login Filament                                |                                                               |        |
| Input transaksi baru                 | Isi form Catat Trade Cepat dan tekan Submit                | Data tersimpan dan notifikasi “Trade Berhasil Dicatat” muncul | Valid  |
| Filter dashboard berdasarkan tanggal | Ubah Tanggal Mulai dan Tanggal Selesai                     | Metrik dan grafik dashboard diperbarui sesuai rentang tanggal | Valid  |
| Konsistensi data Master-Slave        | Bandingkan jumlah baris tabel trades pada Master dan Slave | Jumlah baris pada kedua server identik                        | Valid  |
| Status replikasi                     | Jalankan SHOW SLAVE STATUS pada container Slave            | Slave_IO_Running dan Slave_SQL_Running bernilai Yes           | Valid  |

Tabel 4.1 Hasil Pengujian Black-Box

#### 4.5 Analisis Hasil Pengujian

Pengujian konsistensi data dilakukan dengan melakukan serangkaian operasi INSERT pada Master, kemudian membandingkan jumlah baris tabel trades pada Slave menggunakan perintah `SELECT count(*) FROM trades`. Hasil pengujian menunjukkan bahwa jumlah baris pada Slave selalu identik dengan Master setelah proses replikasi berjalan, membuktikan bahwa mekanisme replikasi asynchronous berjalan dengan baik untuk skala data pada penelitian ini.

Pengujian ketahanan dilakukan dengan menghentikan container Master secara sementara. Hasil pengujian menunjukkan bahwa data pada Slave tetap dapat diakses melalui dashboard monitoring meskipun Master tidak aktif, yang membuktikan bahwa pemisahan server Master dan Slave berhasil meningkatkan ketersediaan (availability) data bagi pengguna dashboard.

Selain itu, dianalisis pula beban komputasi antara mengakses dashboard secara langsung ke Master dibandingkan ke Slave. Hasil analisis menunjukkan bahwa beban CPU pada server Master berkurang ketika query dashboard dialihkan ke Slave, karena Master hanya perlu menangani operasi tulis dan proses replikasi ke Slave, tanpa perlu memproses query analitik yang relatif berat.

Secara keseluruhan, hasil pengujian membuktikan bahwa tujuan penelitian tercapai: replikasi Master-Slave berhasil menjaga konsistensi data secara otomatis, pemisahan beban read-write terbukti mengurangi beban komputasi pada Master, dan seluruh infrastruktur dapat dijalankan secara portabel menggunakan Docker.

## BAB 5 KESIMPULAN DAN SARAN

### 5.1 Kesimpulan

- Sistem replikasi basis data Master-Slave berhasil diimplementasikan menggunakan Docker dan terbukti menjaga konsistensi data jurnal trading secara otomatis antara server Master dan Slave.
- Pemisahan beban kerja antara Master (write) dan Slave (read) terbukti mengurangi beban komputasi pada server utama, sehingga proses pencatatan transaksi tidak terganggu oleh query dashboard.
- Dashboard monitoring berbasis Laravel Filament berhasil menampilkan data historis jurnal trading secara real-time langsung dari server Slave, sebagaimana dibuktikan melalui pengujian fungsional dan verifikasi status replikasi.
- Seluruh infrastruktur, baik basis data maupun aplikasi web, berhasil dijalankan secara portabel menggunakan Docker Compose, sehingga memudahkan reproduksi sistem pada lingkungan atau server yang berbeda.

### 5.2 Saran

- Menambahkan skema multi-slave atau load balancer untuk mengantisipasi peningkatan skala pengguna di masa mendatang.
- Menerapkan mekanisme automatic failover apabila server Master mengalami gangguan, sehingga salah satu Slave dapat dipromosikan menjadi Master secara otomatis.
- Menambahkan monitoring lag replikasi secara real-time pada dashboard, sehingga administrator dapat memantau keterlambatan sinkronisasi data antara Master dan Slave.
- Melakukan pengujian pada skala data dan jumlah pengguna yang lebih besar untuk mengevaluasi performa sistem secara lebih menyeluruh sebelum diterapkan pada lingkungan production.

## DAFTAR PUSTAKA

- Sholihah, I. (2022). Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker. *Majalah Ilmiah Teknologi Elektro*, 21(2). <https://doi.org/10.24843/MITE.2022.v21i02.P08>
- Penerapan Disaster Recovery Plan Pada Database MySQL Menggunakan Metode Master to Master Replication Berbasis Docker. (2024). *Jurnal Komputer Terapan*, 10(1), 78–85.
- Pamungkas, dkk. (2026). Performance Evaluation of a Laravel-based Internship Website using MySQL Master–Slave Replication. *Sistemasi: Jurnal Sistem Informasi*.
- High Availability and Performance of Database in the Cloud — Traditional Master-Slave Replication versus Modern Cluster-based Solutions. (2017).
- Kane, S. P., & Matthias, K. (2023). *Docker: Up & Running: Shipping Reliable Containers in Production*. O'Reilly Media.